Ce que la boucle retient

Première NSI

Travail à distance — variables dans une boucle, parcours d'une chaîne, retour à la machine des nombres

Langages et programmation · Représentation des données

Lundi, la fonction `lire_octet` de la machine des nombres a résisté. Ce n'est pas un hasard : elle demande **trois idées nouvelles en même temps**. Une boucle qui fait évoluer **deux variables**, une boucle `for` qui parcourt **les caractères d'un texte**, et un `if` **à l'intérieur** de la boucle. Aujourd'hui, on apprend ces idées une par une, puis on revient à la machine.

Objectifs

- Suivre, tour par tour, les valeurs d'une variable modifiée dans une boucle.
- Initialiser **avant** la boucle et modifier **dans** la boucle un accumulateur ou un compteur.
- Parcourir une chaîne caractère par caractère avec `for c in texte`.
- Placer un `if` dans une boucle, puis écrire `lire_octet` seul.

Prérequis

- Boucle `for` avec `range`, conditions `if`, fonctions avec `return`.
- Poids des bits d'un octet : 128, 64, 32, 16, 8, 4, 2, 1.
- Un téléphone ou un ordinateur avec Internet : rien à installer. Une feuille et un crayon.

Sommaire

Étape	Travail
A	Une variable qui se souvient : l'accumulateur
B	Deux variables qui changent à chaque tour
C	Parcourir un texte caractère par caractère
D	Un if dans la boucle : le compteur
E	Retour à la machine : lire_octet

Repère

Écrivez votre code dans la zone grise, puis appuyez sur ▸ **Tester** : Python s'exécute dans la page et vérifie votre fonction sur plusieurs exemples. **Exécuter** lance le code sans les tests, pour voir les `print`. Votre code reste enregistré sur cet appareil. Sur téléphone, ⇥ **Indenter** insère 4 espaces. Chaque fonction correcte révèle une lettre du **mot du coffre**.

Attention

Travail personnel, **sans IA**. Les tableaux de suivi se remplissent **sur papier, avant** d'exécuter : c'est là que se construit la compréhension. Si vous bloquez plus de 10 minutes, ouvrez l'indice dans les corrigés en bas de page, puis réessayez.

Mot du coffre

Partie 1 — Une boucle, des variables, un texte

A. Une variable qui se souvient

A1. Suivre la variable à la main

Lisez ce programme **sans l'exécuter**.

```
total = 0
for i in range(4):
    total = total + 3
print(total)
```

1. Recopiez et complétez le tableau : une ligne par tour de boucle, avec la valeur de `total` **à la fin** du tour.

Tour	`i`	`total` à la fin du tour
avant la boucle	—	0
1	0	…
2	…	…
3	…	…
4	…	…

2. Qu'affiche le programme ? Vérifiez avec **Exécuter**.
3. Déplacez la ligne `total = 0` **dans** la boucle, juste au-dessus de `total = total + 3`. Prévoyez le nouvel affichage, puis vérifiez. Pourquoi l'initialisation doit-elle être faite **avant** la boucle ?

> **À retenir**
>
> Un **accumulateur** est une variable **initialisée avant** la boucle et **modifiée dans** la boucle, en partant de sa propre valeur : `total = total + …`. À la fin de chaque tour, il « se souvient » de tout ce qui a été ajouté.

A2. La tirelire

Léa a déjà 20 € dans sa tirelire. Chaque semaine, elle y ajoute 5 €. Écrivez `tirelire(semaines)` qui renvoie la somme dans la tirelire après ce nombre de semaines, **avec une boucle** (sans écrire directement `20 + 5 * semaines`).

```
def tirelire(semaines):
    somme = 20
    ...
    return somme
```

```
tirelire(0) == 20
tirelire(1) == 25
tirelire(4) == 40
tirelire(10) == 70
```

B. Deux variables qui changent à chaque tour

B1. Suivre deux variables à la main

Dans ce programme, **deux** variables évoluent à chaque tour. L'ordre des lignes compte.

```
argent = 0
mise = 1
for i in range(4):
    argent = argent + mise
```

```
    mise = mise * 2
print(argent, mise)
```

1. Complétez sur papier, **sans exécuter** :

Tour	`argent` à la fin du tour	`mise` à la fin du tour
avant la boucle	0	1
1	…	…
2	…	…
3	…	…
4	…	…

2. Qu'affiche le programme ? Vérifiez avec **Exécuter**.
3. Inversez les deux lignes de la boucle (`mise` est doublée **avant** d'être ajoutée). Refaites le tableau, puis vérifiez. Pourquoi le résultat change-t-il ?

B2. La légende de l'échiquier

Selon une légende, le sage qui inventa les échecs demanda une récompense : 1 grain de riz sur la première case de l'échiquier, 2 sur la deuxième, 4 sur la troisième, et ainsi de suite en doublant à chaque case. Écrivez `grains(cases)` qui renvoie le **nombre total** de grains posés sur les premières cases.

Deux variables : `total` (tous les grains déjà posés) et `sur_la_case` (les grains de la case en cours). Inspirez-vous de B1.

```
def grains(cases):
    total = 0
    sur_la_case = 1
    for i in range(cases):
        ...
    return total
```

```
grains(1) == 1
grains(2) == 3
grains(3) == 7
grains(4) == 15
grains(8) == 255
```

Vous avez déjà vu `255` quelque part cette semaine… Où ? Gardez votre réponse pour la partie 2.

C. Parcourir un texte caractère par caractère

C1. `for` sait lire un texte

Jusqu'ici, `for` parcourait les nombres produits par `range`. Il peut aussi parcourir **directement** une chaîne de caractères : à chaque tour, la variable reçoit le caractère suivant, de gauche à droite.

```
for lettre in "NSI":
    print(lettre)

for c in "101":
    print(c, c == "1", c == 1)
```

1. Prévoyez ce qu'affiche la première boucle. Combien de tours fait-elle ?
2. Prévoyez la deuxième boucle, ligne par ligne. Vérifiez avec **Exécuter**.
3. Pourquoi `c == 1` est-il toujours faux ? Que contient vraiment `c` : un nombre ou un texte ?

> **À retenir**
>
> `for c in texte:` fait **un tour par caractère**. `c` est lui-même un texte d'un caractère : `"1"` et non `1`. On le compare donc avec `c == "1"`, entre guillemets.

C2. Compter les caractères

Écrivez `longueur(mot)` qui renvoie le nombre de caractères de `mot` **sans utiliser `len`** : parcourez le texte et ajoutez 1 à chaque tour. La variable `i` de A1 ne servait à rien dans le calcul : ici, c'est pareil pour le caractère lu.

```
def longueur(mot):
    nb = 0
    ...
    return nb
```

```
longueur("NSI") == 3
longueur("octet") == 5
longueur("") == 0
longueur("01001101") == 8
```

D. Un `if` dans la boucle

D1. Ne compter que certains caractères

Maintenant, on ne compte **que** les caractères qui vérifient une condition. Le `if` est **dans** la boucle (décalé une fois), la ligne qui compte est **dans** le `if` (décalée deux fois). Écrivez `compte(mot, lettre)` qui renvoie le nombre de fois où `lettre` apparaît dans `mot`.

```
def compte(mot, lettre):
    nb = 0
    for c in mot:
        ...
    return nb
```

```
compte("ananas", "a") == 3
compte("ananas", "n") == 2
compte("NSI", "z") == 0
compte("", "a") == 0
```

D2. Combien de bits allumés ?

Écrivez `nb_uns(bits)` qui renvoie le nombre de `"1"` dans une écriture binaire. Par exemple, `"01001101"` contient quatre `1`.

```
def nb_uns(bits):
    ...
```

```
nb_uns("01001101") == 4
nb_uns("00000000") == 0
nb_uns("11111111") == 8
nb_uns("1") == 1
```

Partie 2 — Retour à la machine des nombres

E1. Lire quatre bits à la main

On veut lire un **quartet** (4 bits) : les poids sont 8, 4, 2, 1. Voici la fonction complète. Elle réunit tout ce qui précède : un accumulateur `total`, une seconde variable `poids` qui change à chaque tour, un parcours du texte et un `if`.

```
def lire_quartet(bits):
    total = 0
    poids = 8
    for bit in bits:
        if bit == "1":
            total = total + poids
        poids = poids // 2
    return total

print(lire_quartet("1011"))
```

`poids // 2` est la **division entière** : `8 // 2` vaut `4`, `1 // 2` vaut `0`. Le poids reste un nombre entier.

1. Complétez sur papier, **sans exécuter**, pour `lire_quartet("1011")` :

Tour	`bit` lu	`bit == "1"` ?	`total` à la fin du tour	`poids` à la fin du tour
avant la boucle	—	—	0	8
1	`"1"`	…	…	…
2	…	…	…	…
3	…	…	…	…
4	…	…	…	…

2. Quelle valeur la fonction renvoie-t-elle ? Vérifiez à la main : $1011_2 = 8 + 2 + 1$. Puis avec **Exécuter**.
3. La ligne `poids = poids // 2` est-elle **dans** le `if` ou seulement **dans** la boucle ? Que se passerait-il si on la décalait d'un cran de plus, dans le `if` ? Testez avec `"0011"`.

E2. Lire un octet

Écrivez maintenant `lire_octet(bits)` **sans regarder** la fonction précédente : un octet a huit bits, le premier caractère est le bit de poids 128. N'utilisez pas `int(bits, 2)`. Si vous bloquez, relisez votre tableau de E1 plutôt que le code.

```
def lire_octet(bits):
    ...
```

```
lire_octet("00000000") == 0
lire_octet("00000001") == 1
lire_octet("10000000") == 128
lire_octet("01001101") == 77
lire_octet("10101100") == 172
lire_octet("11111111") == 255
```

Retour à la question de B2 : `lire_octet("11111111")` et `grains(8)` valent tous deux `255`. Expliquez en une phrase pourquoi ce n'est pas une coïncidence.

Pour la reprise en classe

Rien à envoyer. Gardez sur votre feuille le **mot du coffre**, vos tableaux de suivi et, en trois phrases, la réponse à ces questions sur `lire_octet` : que contient `total` ? que contient `poids` ? à quoi sert le `if` ? Nous repartirons de vos réponses au retour des vacances.

Corrigés

Première NSI

Travail à distance — variables dans une boucle, parcours d'une chaîne, retour à la machine des nombres

Langages et programmation · Représentation des données

Ouvrez d'abord l'indice, réessayez, puis seulement la solution.

A1

Tour	`i`	`total`
avant	—	0
1	0	3
2	1	6
3	2	9
4	3	12

Le programme affiche `12`. Si `total = 0` est dans la boucle, la variable est remise à zéro à chaque tour : on affiche `3`. Elle « oublie » ce qui a été ajouté avant.

A2

Indice : la boucle doit faire `semaines` tours ; à chaque tour, on ajoute 5 à `somme`.

```
def tirelire(semaines):
    somme = 20
    for i in range(semaines):
        somme = somme + 5
    return somme
```

B1

Tour	`argent`	`mise`
avant	0	1
1	1	2
2	3	4
3	7	8
4	15	16

Affichage : `15 16`. Avec les lignes inversées, la mise est doublée avant d'être ajoutée : `argent` vaut 2, 6, 14 puis 30, et l'on affiche `30 16`. Dans une boucle, chaque ligne utilise les valeurs **laissées par la ligne précédente**.

B2

Indice : c'est exactement B1, avec `total` à la place d'`argent` et `sur_la_case` à la place de `mise`.

```
def grains(cases):
    total = 0
    sur_la_case = 1
    for i in range(cases):
        total = total + sur_la_case
```

```
        sur_la_case = sur_la_case * 2
    return total
```

`255` est la valeur maximale d'un octet.

C1

La première boucle fait 3 tours et affiche `N`, `S`, `I` sur trois lignes. La deuxième affiche :

```
1 True False
0 False False
1 True False
```

`c` contient le **texte** `"1"` ou `"0"`, jamais le nombre `1` : la comparaison avec un nombre est toujours fausse.

C2

Indice : `for c in mot:`, puis `nb = nb + 1` dans la boucle. On n'utilise pas `c`.

```
def longueur(mot):
    nb = 0
    for c in mot:
        nb = nb + 1
    return nb
```

D1

Indice : la condition est `c == lettre`. Seule la ligne `nb = nb + 1` est dans le `if`.

```
def compte(mot, lettre):
    nb = 0
    for c in mot:
        if c == lettre:
            nb = nb + 1
    return nb
```

D2

Indice : c'est `compte` avec la lettre `"1"`. Attention aux guillemets : `bit == 1` est toujours faux.

```
def nb_uns(bits):
    nb = 0
    for bit in bits:
        if bit == "1":
            nb = nb + 1
    return nb
```

E1

Tour	`bit`	`bit == "1"` ?	`total`	`poids`
avant	—	—	0	8
1	`"1"`	oui	8	4
2	`"0"`	non	8	2
3	`"1"`	oui	10	1
4	`"1"`	oui	11	0

La fonction renvoie `11`. La ligne `poids = poids // 2` est dans la boucle mais **pas** dans le `if` : le poids diminue à chaque position, que le bit soit allumé ou non. Si on la place dans le `if`, un `0` ne fait plus avancer le poids : `lire_quartet("0011")` renverrait `8 + 4 = 12` au lieu de `3`.

E2

Indice : partez de `lire_quartet` en ne changeant qu'un nombre.

```
def lire_octet(bits):
    total = 0
    poids = 128
    for bit in bits:
        if bit == "1":
            total = total + poids
        poids = poids // 2
    return total
```

`11111111` additionne tous les poids : `128 + 64 + … + 1`. L'échiquier additionne les mêmes nombres, de 1 à 128, sur huit cases : les deux valent `255`, soit $2^8 - 1$.

Mot du coffre : **TRACE**.